

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

GUÍA DE PRÁCTICA EXPERIMENTAL

1 DATOS GENERALES

CARRERA: SOFTWARE (REDISEÑO)

ASIGNATURA: APLICACIONES DISTRIBUIDAS [20701]

NIVEL / PARALELO: 7MO NIVEL – A

N.º DE ESTUDIANTES:

PROFESOR: GUERRERO ULLOA GLEISTON CICERON

PERIODO ACADÉMICO: 2026-2027 PPA

FECHA: 29-04-2026

TIPO DE GUÍA: PRÁCTICA EXPERIMENTAL

TIPO DE UBICACIÓN: INTERNA

TIPO AULA/LABORATORIO: LABORATORIOS

AULA/LABORATORIO: FCCDD-LAB-TIC-201

UBICACIÓN: CAMPUS CENTRAL

TIPO DE APRENDIZAJE: Aplicación de contenidos procedimentales

N. HORAS: 9

2 DATOS ACADÉMICOS

Resultado de aprendizaje

Trabaja colaborativamente en equipos multidisciplinarios para planificar, desarrollar y entregar proyectos de software distribuido en ciclos iterativos, aplicando metodologías ágiles.

Tema de la práctica

Implementación de la capa de calidad en la aplicación distribuida del PFC: pipeline CI/CD con GitHub Actions, observabilidad distribuida con la tríada métricas-logs-trazas (Prometheus + Grafana), pruebas automatizadas (unitarias, integración, carga), y evaluación de calidad según ISO/IEC 25010:2023.

3 OBJETIVO

Objetivo General:

Implementar un pipeline CI/CD completo y un sistema de observabilidad distribuida para el PFC, evaluar la calidad del sistema con ISO/IEC 25010:2023, y documentar el proyecto completo del período en un documento LaTeX de calidad académica total que sirva como portafolio profesional.

Objetivos Específicos:

1. Implementar un pipeline CI/CD con GitHub Actions: pruebas automatizadas, build Docker y análisis de calidad de código.
2. Configurar observabilidad: logging estructurado JSON, métricas Prometheus y dashboard Grafana Cloud.
3. Ejecutar y documentar pruebas unitarias (cobertura $\geq 70\%$), de integración y de carga (Locust).
4. Evaluar el sistema contra 5 atributos de calidad de ISO/IEC 25010:2023 con métricas reales.

4 FUNDAMENTO TEÓRICO

Esta práctica ES el documento final del PFC (Entrega 4). Es ACUMULATIVO: incluye todas las secciones de las Entregas 1-3 ACTUALIZADAS, más las secciones nuevas de la Unidad 5. Mínimo 20 páginas de contenido. Bibliografía total: ≥ 12 referencias IEEE verificadas.

5.1 Arquitectura en Capas y Patrones de Diseño Distribuido

Desarrollar: patrón N-capas en sistemas distribuidos, principios SOLID aplicados a microservicios, patrones de diseño GoF más usados en SD (Repository, Factory Method, Proxy, Observer, Strategy, Decorator), inyección de dependencias y frameworks IoC, y cómo estos patrones reducen el acoplamiento.

Referencias bibliográficas mínimas [IEEE]:

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

[2] S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ch. 12-15.

5.2 Pruebas en Sistemas Distribuidos

Desarrollar: pirámide de pruebas adaptada a microservicios (unitarias, integración, contrato, extremo a extremo), pruebas de contrato con Pact (Consumer-Driven Contract Testing), pruebas de carga con Locust o k6, cobertura de código como métrica de calidad, y herramientas de análisis estático (SonarCloud, Pylint, ESLint).

Referencias bibliográficas mínimas [IEEE]:

[3] T. Winters, T. Manshreck, and H. Wright, Software Engineering at Google. Sebastopol, CA, USA: O'Reilly Media, 2020. ISBN: 978-1-4920-8276-8.

[4] C. Richardson, Microservices Patterns. Shelter Island, NY, USA: Manning Publications, 2018, ch. 11, pp. 357-402.

5.3 DevOps: CI/CD con GitHub Actions

Desarrollar: concepto DevOps y cultura desarrollo-operaciones, pipeline CI/CD (etapas: build, test, lint, publish, deploy), sintaxis YAML de GitHub Actions (triggers, jobs, steps, marketplace actions), estrategias de despliegue (blue-green, canary, rolling update), y integración de cobertura de código en el pipeline.

Referencias bibliográficas mínimas [IEEE]:

[5] G. Kim, J. Humble, P. Debois, and J. Willis, The DevOps Handbook, 2nd ed. Portland, OR, USA: IT Revolution Press, 2021. ISBN: 978-1-950508-43-7.

[6] J. Humble and D. Farley, Continuous Delivery. Boston, MA, USA: Addison-Wesley, 2010. ISBN: 978-0-321-60191-8.

5.4 Observabilidad Distribuida y Calidad ISO/IEC 25010:2023

Desarrollar: la tríada de observabilidad (logs estructurados JSON, métricas Prometheus, trazas con OpenTelemetry), modelo de datos Prometheus (counter, gauge, histogram, summary), visualización con Grafana (paneles, alertas), rastreo distribuido, y el modelo de calidad ISO/IEC 25010:2023 con 8 características y 31 subcaracterísticas. Evaluar el sistema propio contra 5 características con métricas concretas.

Referencias bibliográficas mínimas [IEEE]:

[7] International Organization for Standardization, 'ISO/IEC 25010:2023 — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model,' ISO, Geneva, Switzerland, 2023.

[8] C. Richardson, Microservices Patterns. Shelter Island, NY, USA: Manning Publications, 2018.

5 MATERIALES, EQUIPOS, INSUMOS Y REACTIVOS

Materiales

- LaTeX completo: paquetes minted o listings, pgfplots (para gráficas), todos los
- SonarCloud o Codecov (gratuitos para repositorios públicos)
- Cuenta gratuita en Grafana Cloud (grafana.com) y GitHub
- Grafana Agent (Docker: grafana/agent:latest)+Prometheus (prom/prometheus:latest)
- Locust 2.x: `pip install locust` para pruebas de carga
- pytest + pytest-cov — O — JUnit 5 + JaCoCo — O — Jest + Istanbul (según tecnolog
- Docker Desktop 4.x con Docker Compose.

Equipos

- Con 8 GB RAM: ajustar mem_limit en docker-compose.
- para todos los contenedores: microservicios + Prometheus + Grafana Agent
- Computadora con ≥16 GB RAM

Insumos y reactivos

- Conexión a Internet estable para Grafana Cloud y GitHub Actions.

6 PROCEDIMIENTOS

Paso 1 Pruebas Automatizadas

- Implementar pruebas unitarias para la capa de lógica de negocio de cada microservicio (pytest / JUnit / Jest). Objetivo: cobertura $\geq 70\%$.
- Implementar pruebas de integración: ≥ 3 flujos end-to-end que verifiquen comunicación entre microservicios a través del API Gateway.
- Configurar reporte de cobertura: `pytest --cov=src --cov-report=xml` → subir a Codecov o SonarCloud (gratuito para proyectos públicos).
- Documentar: tipos de pruebas en Istlisting, resultados de cobertura en tabla booktabs, análisis de qué queda fuera del alcance.

Paso 2 Pipeline CI/CD con GitHub Actions

- Crear `.github/workflows/ci-cd.yml` con jobs: (a) test: ejecuta pruebas + cobertura, (b) lint: análisis estático (Pylint/ESLint/Checkstyle), (c) build: construye imagen Docker, (d) integration-test: levanta docker-compose y ejecuta pruebas de integración.
- Triggers: push a main y pull_request.
- Demostrar en vivo: hacer push con fallo intencional → mostrar error → corregir → pipeline verde.
- El pipeline VERDE es condición necesaria para la demostración.

△ **Nota:** El archivo `ci-cd.yml` completo debe incluirse en Istlisting en el documento LaTeX.

Paso 3 Observabilidad Distribuida

- Añadir logging estructurado JSON en todos los microservicios: timestamp, servicio, método, ruta, código HTTP, tiempo de respuesta (ms).
- Exponer métricas en `/metrics` de cada microservicio: `request_count_total` (counter), `request_duration_seconds` (histogram), `active_connections` (gauge).
- Configurar Prometheus en docker-compose para scraping cada 15 segundos de los tres microservicios.
- Conectar Grafana Cloud (gratuito: grafana.com) mediante Grafana Agent. Dashboard: tasa de requests/seg, latencia P50/P95/P99, tasa de errores 4xx/5xx.
- Ejecutar prueba de carga con Locust (50 usuarios, 60 segundos) y capturar el dashboard durante la prueba.

Paso 4 Evaluación ISO/IEC 25010:2023

- Evaluar 5 características con métricas reales: (a) Disponibilidad: uptime $\geq 99.5\%$ durante 1h continua; (b) Rendimiento: latencia P95 $< 500\text{ms}$ con 50 usuarios; (c) Fiabilidad: tasa de errores 5xx $< 1\%$; (d) Mantenibilidad: cobertura $\geq 70\%$ y complejidad ciclomática < 10 ; (e) Seguridad: todos los endpoints protegidos retornan 401 sin token.
- Tabla booktabs en LaTeX: Característica | Métrica | Valor medido | Valor objetivo | Estado (✓/✗) | Plan de mejora.
- La evaluación debe ser honesta — documentar las deficiencias con plan de mejora concreto.

Paso 5 Documento LaTeX Final y Defensa Oral

- El documento acumulativo incluye TODAS las secciones del PFC actualizadas + secciones U5: diseño de pruebas, pipeline CI/CD en Istlisting YAML, observabilidad con dashboard como figura, evaluación ISO/IEC 25010 con tabla, reflexión ética ($\geq \frac{1}{2}$ página, referenciando ACM/IEEE-CS Code of Ethics), conclusiones individuales (≥ 200 palabras c/u), bibliografía ≥ 12 referencias IEEE.
- La defensa (15 min: 10 exposición + 5 preguntas) demuestra en vivo: sistema funcionando, pipeline verde en GitHub, dashboard Grafana con datos reales.
- Todos los integrantes deben responder preguntas técnicas sobre cualquier parte del sistema.

7 BIBLIOGRAFÍA

- B. Burns, Designing Distributed Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2025. ISBN: 978-1-09-815635-0.
- S. Newman, Building Microservices, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021. ISBN: 978-1-4920-3402-5.

- T. Winters, T. Manshreck, and H. Wright, Software Engineering at Google. Sebastopol, CA, USA: O'Reilly Media, 2020. ISBN: 978-1-4920-8276-8.
- International Organization for Standardization, 'ISO/IEC 25010:2023,' ISO, Geneva, Switzerland, 2023.
- C. Richardson, Microservices Patterns. Shelter Island, NY, USA: Manning Publications, 2018. ISBN: 978-1-617-29454-1.
- J. Humble and D. Farley, Continuous Delivery. Boston, MA, USA: Addison-Wesley, 2010. ISBN: 978-0-321-60191-8.
- G. Kim, J. Humble, P. Debois, and J. Willis, The DevOps Handbook, 2nd ed. Portland, OR, USA: IT Revolution Press, 2021. ISBN: 978-1-950508-43-7.
- E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.

8 ANEXOS

- ¿Cómo garantiza el pipeline CI/CD que el código desplegado cumple los atributos de calidad? ¿Qué quality gates adicionales agregarías?
- Analiza los resultados de la prueba de carga con Locust: ¿hay degradación con 50 usuarios? ¿Cuál microservicio es el cuello de botella? ¿Por qué?
- ¿Qué características ISO/IEC 25010 no pudo cumplir el sistema? Diseña un plan técnico concreto para alcanzarlas en producción real.
- Reflexión ética: ¿qué responsabilidades tienen como futuros Ingenieros de Software frente a los datos de los usuarios del sistema del PFC? Analiza desde el Código de Ética ACM/IEEE-CS (1999).

9 RESULTADOS DE ESTUDIANTES

EQUIPO ACC

ALVAREZ PARRAGA JEREMY ALEXIS
CANDO MORENO ROBINSON RODRIGO
CARPIO MENDOZA CARLOS JOSE
PACHECO CARDENAS CRISTHIAN DANIEL

Resultados obtenidos

Se completó la pirámide de pruebas del sistema: contratos Pact verificados contra el backend real, pruebas end-to-end con Playwright en tres navegadores, y pruebas de carga con Locust. Se construyó un pipeline de CI/CD de siete jobs en GitHub Actions que, al ejecutarse, reveló tres fallas reales: imágenes Docker con nombre en mayúsculas, falta de permiso de escritura para GHCR, y un error del script gradlew al interpretar sus opciones de memoria. Las tres se diagnosticaron con los logs del pipeline y se corrigieron. También se instrumentó observabilidad completa (métricas, logs, trazas) y se evaluó el sistema contra ISO/IEC 25010.

Conclusión

Esta actividad demostró que un pipeline de CI/CD solo se valida corriéndolo en el entorno real, no leyendo el YAML: las tres fallas encontradas (mayúsculas en las imágenes, permisos de GHCR, y el bug de gradlew) eran invisibles en pruebas locales y solo aparecieron en GitHub Actions. Diagnosticar con los logs reales, en vez de asumir la causa, evitó arreglos equivocados. La observabilidad y la evaluación ISO/IEC 25010 reforzaron esa misma disciplina: medir de verdad, reportando también lo que no cumplió el umbral. La calidad de un sistema distribuido se demuestra con evidencia verificable, no con documentación que asume que todo funciona.

EQUIPO AGLS

AUCATOMA CELORIO JHINSON STALYN
GAIBOR RODRIGUEZ JEREMY RUPERTO
LOZANO MORALES JOSE ALEJANDRO
SANCHEZ PILALOA ANDY PAUL

Resultados obtenidos

Se completó el sistema distribuido TiendaTech con sus dos clientes obligatorios sobre la misma API. La SPA en React 18 y TypeScript expone doce rutas —catálogo, detalle de producto, armado de PC, carrito, pago, factura, autenticación, cuenta, panel

de trabajador y administración—, superando el mínimo de cinco de la rúbrica, con sesión JWT almacenada exclusivamente en sessionStorage, ruta de administración protegida por rol y expiración automática por inactividad. El empaquetado se resolvió con un Dockerfile multi-stage de tres etapas que deja la imagen final libre de herramientas de compilación. En el backend, cuatro de los seis microservicios Java cuentan con pruebas unitarias JUnit 5, y pedidos-service incorpora la única prueba de integración real, que levanta un CockroachDB efímero mediante Testcontainers. El pipeline de GitHub Actions ejecuta lint, pruebas unitarias e instrumentadas del cliente Android con publicación del APK, verificación con Testcontainers y pruebas del servicio Python, además de publicar automáticamente las ocho imágenes multi-arquitectura del sistema

Conclusión

La Entrega 4 cierra el ciclo del PFC con un sistema distribuido funcional en sus tres capas y con automatización real de construcción y publicación de artefactos. Los resultados confirman que la arquitectura de microservicios heredada de las entregas previas sostiene sin cambios de contrato a dos clientes heterogéneos. No obstante, la evaluación evidencia brechas que se documentan de forma honesta en lugar de omitirse: ausencia de internacionalización en la SPA, falta de linter y de framework de pruebas en el frontend, dos microservicios sin cobertura pese a concentrar la lógica de negocio más sensible, y un pipeline que cubre parcialmente los siete jobs exigidos. Estas limitaciones definen la agenda inmediata de trabajo futuro del equipo

EQUIPO BCEL

BEDON VITERI KEYLA BETZABE
CASTRO LOPEZ PEDRO LEONARDO
EMANUEL PINO JULIANA ROMINA
LUNA MORA ERNESTO GREGORY

Resultados obtenidos

La implementación de las actividades asignadas al microservicio Docente permitió fortalecer la calidad, observabilidad y confiabilidad del sistema distribuido SGA. Se incorporaron pruebas automatizadas sobre los principales procesos del módulo, junto con mecanismos de cobertura y análisis estático del código. También se configuró el registro estructurado en formato JSON y la exposición de métricas para Prometheus, facilitando el seguimiento de solicitudes, tiempos de respuesta y conexiones activas. Asimismo, se verificó el funcionamiento de endpoints protegidos, la construcción mediante Docker y la preparación del servicio para integrarse al pipeline CI/CD, las pruebas end-to-end y las pruebas de carga del sistema completo.

Conclusión

El trabajo realizado en el microservicio Docente demuestra la importancia de complementar el desarrollo funcional con prácticas orientadas a calidad, seguridad y observabilidad. La incorporación de pruebas automatizadas, cobertura, análisis estático, métricas y registros estructurados permite detectar errores con mayor rapidez y mantener un comportamiento más controlado durante la ejecución. Además, la integración con Docker y el pipeline CI/CD facilita validar los cambios antes de incorporarlos al sistema distribuido. Estas mejoras también aportan evidencias objetivas para la evaluación mediante ISO/IEC 25010:2023 y para las pruebas de carga posteriores. Como resultado, el microservicio queda mejor preparado para operar de manera estable, medible y mantenible, reduciendo riesgos durante la integración con los demás componentes del SGA y futuras actualizaciones del sistema.

FUVV

FARINANGO GUANDINANGO FREDDY VLADIMIR
URBINA ROMERO ISAIAS ABRAHAM
VILLAMARIN CUENCA IVAN ANDRES
VINUEZA SANCHEZ HAROLD NICOLAS

Resultados obtenidos

Durante el desarrollo de la práctica se implementó un proceso integral de integración y entrega continua, acompañado de pruebas automatizadas y mecanismos básicos de observabilidad para los servicios del sistema. Se configuraron flujos de trabajo que permiten compilar, validar y verificar automáticamente el código ante cada cambio realizado en el repositorio. También se comprobaron los componentes principales, la comunicación entre servicios y el correcto funcionamiento de las funcionalidades existentes. La incorporación de métricas, registros y controles facilitó la identificación de errores y el seguimiento del comportamiento general de la aplicación. Finalmente, el equipo fortaleció el trabajo colaborativo, organizó mejor las versiones del proyecto y obtuvo una solución más estable, verificable y preparada para futuras mejoras, correcciones y despliegues controlados posteriores.

Conclusión

La práctica permitió comprender la importancia de integrar desarrollo, pruebas y observabilidad dentro de un mismo proceso de trabajo. La automatización de las verificaciones reduce errores humanos, mejora la calidad del software y proporciona mayor confianza al momento de incorporar cambios en el repositorio. Asimismo, las pruebas permiten detectar fallos oportunamente, mientras que los registros, métricas y controles ayudan a conocer el estado de la aplicación. El desarrollo colaborativo también favoreció la organización del equipo y la distribución de responsabilidades. En conclusión, la aplicación conjunta de estas prácticas constituye una base sólida para mantener el sistema estable, seguro y disponible. Además, facilita futuras actualizaciones, correcciones y despliegues controlados, contribuyendo a que el proyecto pueda evolucionar de manera ordenada, confiable y sostenible.

10 RESPONSABLES

GUERRERO ULLOA GLEISTON CICERON
CI: 0913531752
DOCENTE RESPONSABLE

PONCE ORDOÑEZ JESSICA ALEXANDRA
CI: 1205316456
COORDINADOR DE CARRERA